WORKSHOP 2

REPORT

Spotify Tracks Dataset — Popularity Analysis & Prediction

Submitted by

Bibek Karki

Pravin Sapkota

1) Data Source Identification

Dataset Overview

The dataset used is the Spotify Tracks CSV — a structured file containing audio features and metadata for 50,000 Spotify tracks sourced from Google Drive.

Attribute	Details
Source	spotify_tracks.csv — Google Drive / Colab
Records	50,000 tracks
Features	21 columns (numeric, categorical, boolean)
Missing Values	None — fully clean
Year Range	2000 – 2024

2) Data Exploration, Transformation & Model Training

2.1 Loading the Dataset

```
df = pd.read_csv('/content/drive/MyDrive/Big Data/spotify_tracks.csv')
print(df.head())
```

Output:

```
                 track_id                     track_name     artist_name
0  P3fAbnFbmOHnKYaXRvj7uf  One Dance (Acoustic Version)  Alex Rodriguez
1  M2wleOV911xCZkwPRQeNHp          Forever Song (Remix)     Desert Wind
2  4JSnE2NiiUHUAKw9iEU1jj                 Last Mountain    The Midnight
3  2UVvsjaSS8VFgM0Fmxk754           Falling Star (Live)    Phantom Keys
4  EeVVhDIq2AnHTmt9OBGhnu   Rising Moon (feat. someone)     Desert Wind

   release_year  genre  popularity  duration_ms  explicit  danceability ...
0          2024  metal          14       189042      True      0.427723 ...
1          2019   rock          11       186805      True      0.448634 ...
2          2022  k-pop          23       121814     False      0.707923 ...
3          2024  latin          34       216049     False      0.846237 ...
4          2010  latin          31       156170     False      0.943720 ...
[5 rows x 21 columns]
```

2.2 Dataset Info & Shape

```
print(df.shape)
print(df.info())
```

Output:

```
(50000, 21)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   track_id          50000 non-null  object
 1   track_name        50000 non-null  object
 2   artist_name       50000 non-null  object
 3   album_name        50000 non-null  object
 4   release_year      50000 non-null  int64
 5   genre             50000 non-null  object
 6   popularity        50000 non-null  int64
 7   duration_ms       50000 non-null  int64
 8   explicit          50000 non-null  bool
 9   danceability      50000 non-null  float64
10   energy            50000 non-null  float64
11   loudness          50000 non-null  float64
12   speechiness       50000 non-null  float64
13   acousticness      50000 non-null  float64
14   instrumentalness  50000 non-null  float64
15   liveness          50000 non-null  float64
16   valence           50000 non-null  float64
17   tempo             50000 non-null  float64
18   key               50000 non-null  int64
19   mode              50000 non-null  int64
```

```
20   time_signature    50000 non-null  int64
dtypes: bool(1), float64(9), int64(6), object(5)
memory usage: 7.7+ MB
```

2.3 Missing Values Check

```
print(df.isnull().sum())
```

Output:

```
track id            0    track name          0    artist name         0
album_name          0    release_year        0    genre               0
popularity          0    duration_ms         0    explicit            0
danceability        0    energy              0    loudness            0
speechiness         0    acousticness        0    instrumentalness    0
liveness            0    valence             0    tempo               0
key                 0    mode                0    time signature      0
dtype: int64  →  No missing values in any column.
```

2.4 Descriptive Statistics

```
print(df.describe())
```

Stat	popularity	danceability	energy	loudness	valence	tempo
count	50000	50000	50000	50000	50000	50000
mean	23.60	0.627	0.659	-7.44	0.581	116.26
std	17.91	0.208	0.213	4.31	0.184	23.71
min	-5.00	0.000	0.000	-27.80	0.000	50.00
50%	19.00	0.651	0.683	-6.91	0.581	115.33
max	100.00	1.000	1.000	0.000	1.000	220.00

2.5 Genre Average Audio Features

```
genre_avg = df.groupby('genre')[['danceability','energy','loudness',

'speechiness','acousticness','instrumentalness','liveness','valence','tempo']].mean
()
print(genre_avg)
```

Output (selected rows):

```
               danceability    energy   loudness  speechiness  acousticness
genre
ambient            0.250055  0.185659 -17.807170     0.036243      0.552319
blues              0.517610  0.506806 -10.950936     0.051406      0.547890
classical          0.252256  0.250483 -15.020991     0.044116      0.873432
country            0.601050  0.617833  -7.536790     0.042050      0.480167
electronic         0.780027  0.841001  -5.608318     0.052213      0.078852
hip-hop            0.816443  0.679998  -6.085443     0.219806      0.112541
k-pop              0.776448  0.776478  -5.630787     0.070606      0.127504
```

```
latin              0.814814  0.744783  -6.478239     0.071549      0.182495
metal              0.344240  0.909654  -4.148428     0.061768      0.074094
pop                0.719523  0.701026  -5.547990     0.060307      0.154660
rock               0.498396  0.817447  -5.106825     0.051226      0.130852
```

2.6 Average Popularity by Genre

```
genre_pop = df.groupby('genre')['popularity'].mean().sort_values(ascending=False)
print(genre_pop)
```

Output:

```
genre
rock             24.345763
indie            24.047407
jazz             23.827497
ambient          23.807777
reggae           23.724856
folk             23.709406
electronic       23.666303
punk             23.565353
blues            23.552758
pop              23.526525
k-pop            23.523956
metal            23.120727
drum-and-bass    22.702174
Name: popularity, dtype: float64
```

2.7 Visualization Charts

Chart 1: Distribution of Song Popularity

```
plt.figure(figsize=(8,5))
sns.histplot(df['popularity'], bins=30)
plt.title('Distribution of Song Popularity')
plt.show()
```

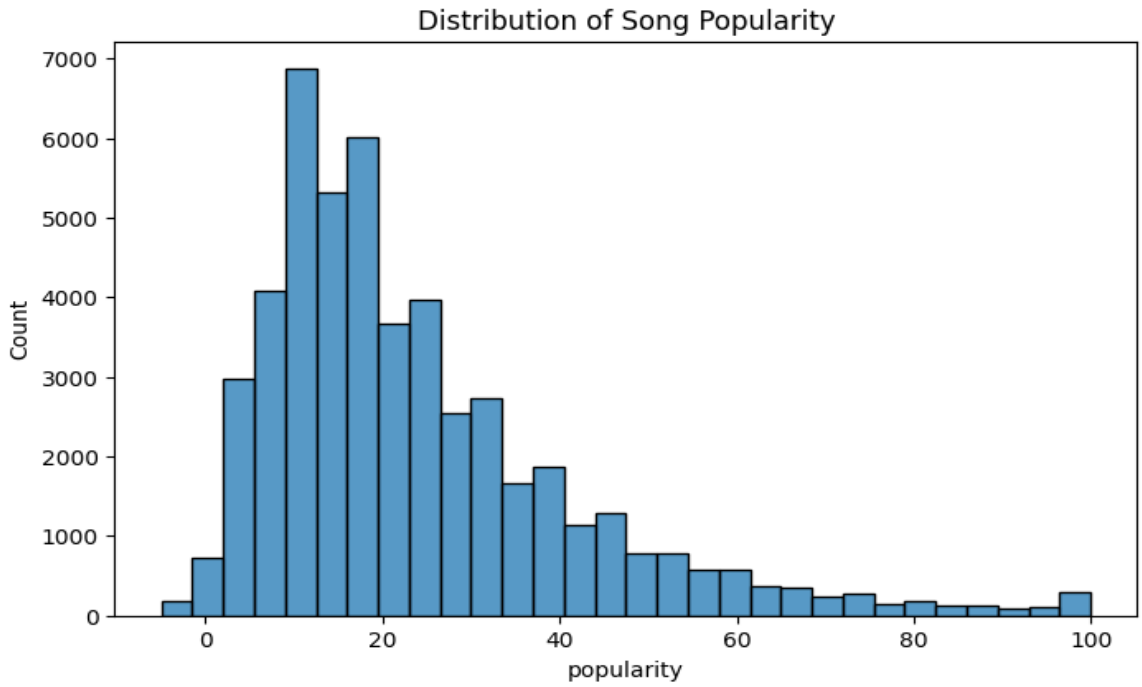

Figure 1 — Distribution of Song Popularity (sns.histplot)

Chart 2: Distribution of Song Popularity (Matplotlib)

```
plt.hist(df['popularity'], bins=30)
plt.title('Distribution of Song Popularity')
plt.xlabel('Popularity') | plt.ylabel('Number of Songs')
plt.show()
```

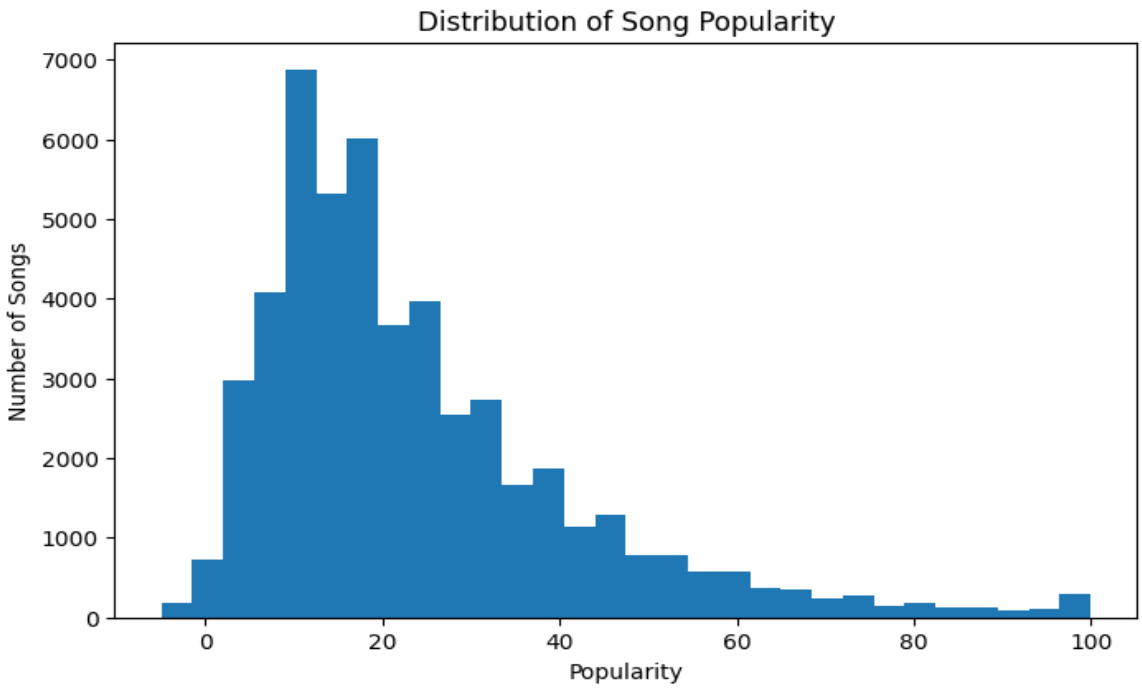

Figure 2 — Popularity Distribution (matplotlib.hist)

Chart 3: Top 10 Genres by Track Count

```
df['genre'].value_counts().head(10).plot(kind='bar')
plt.title('Top 10 Genres') | plt.show()
```

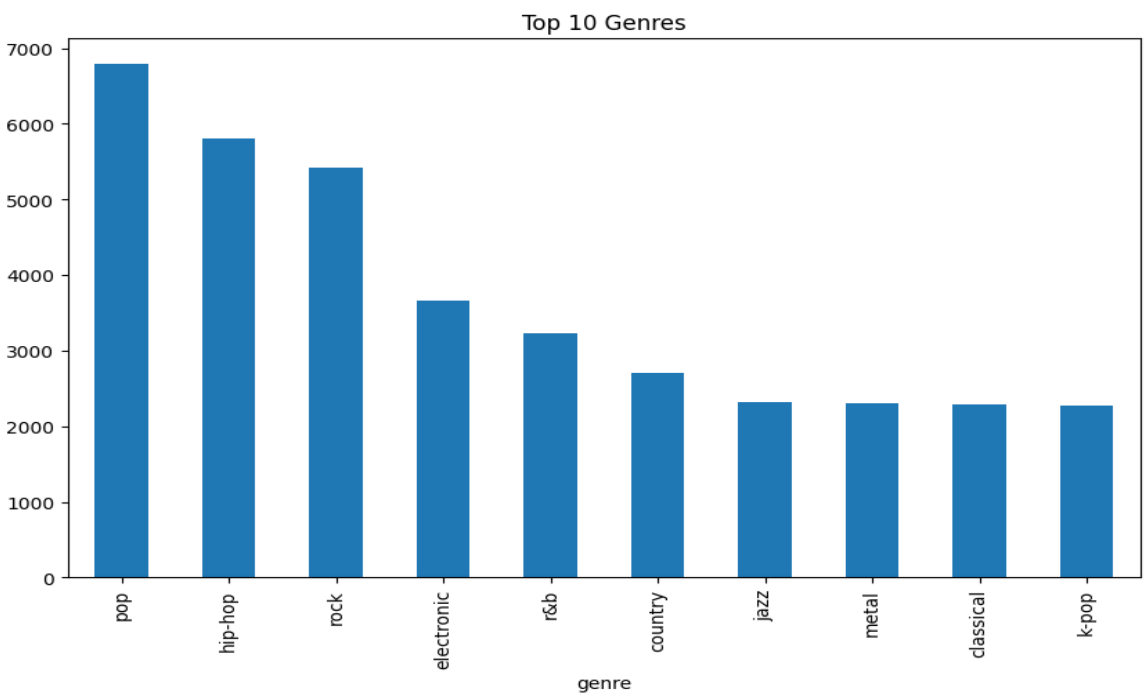

Figure 3 — Top 10 Genres by Number of Tracks

Chart 4: Average Energy per Genre

```
genre_avg['energy'].sort_values().plot(kind='barh', figsize=(10,6))
plt.title('Average Energy per Genre')
plt.show()
```

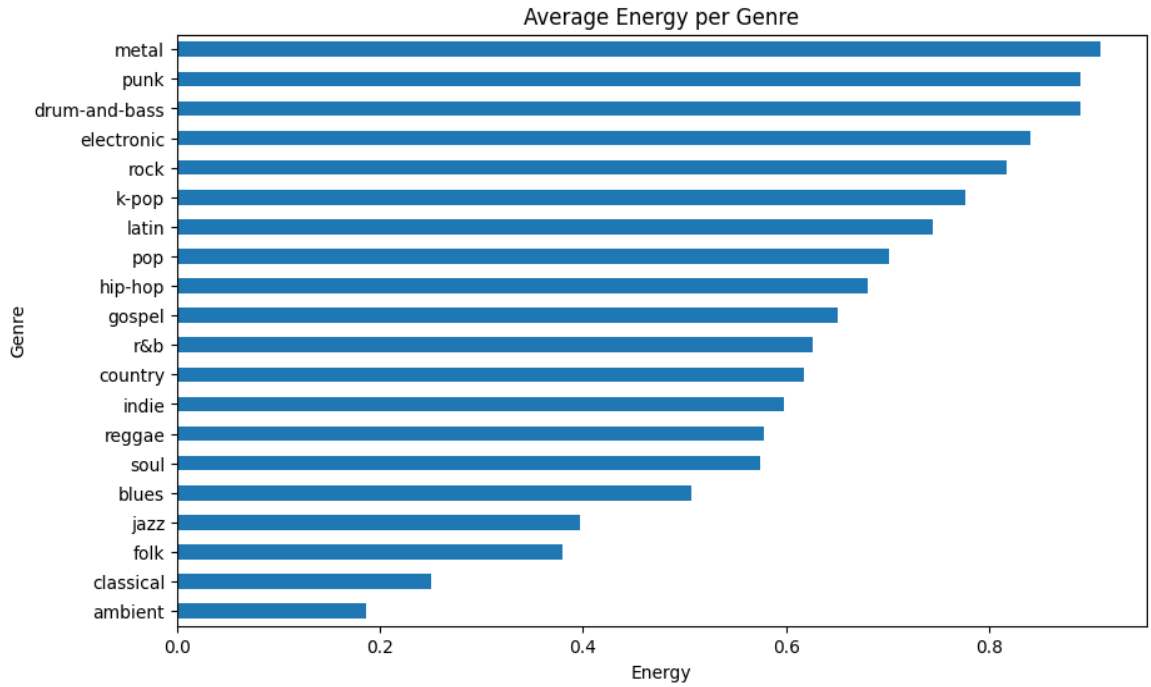

Figure 4 — Average Audio Energy per Genre

Chart 5: Average Popularity per Genre (Bar Chart)

```
genre_popularity =
df.groupby('genre')['popularity'].mean().sort_values(ascending=False)
genre_popularity.head(10).plot(kind='bar', figsize=(10,6))
plt.title('Average Popularity per Genre') | plt.show()
```

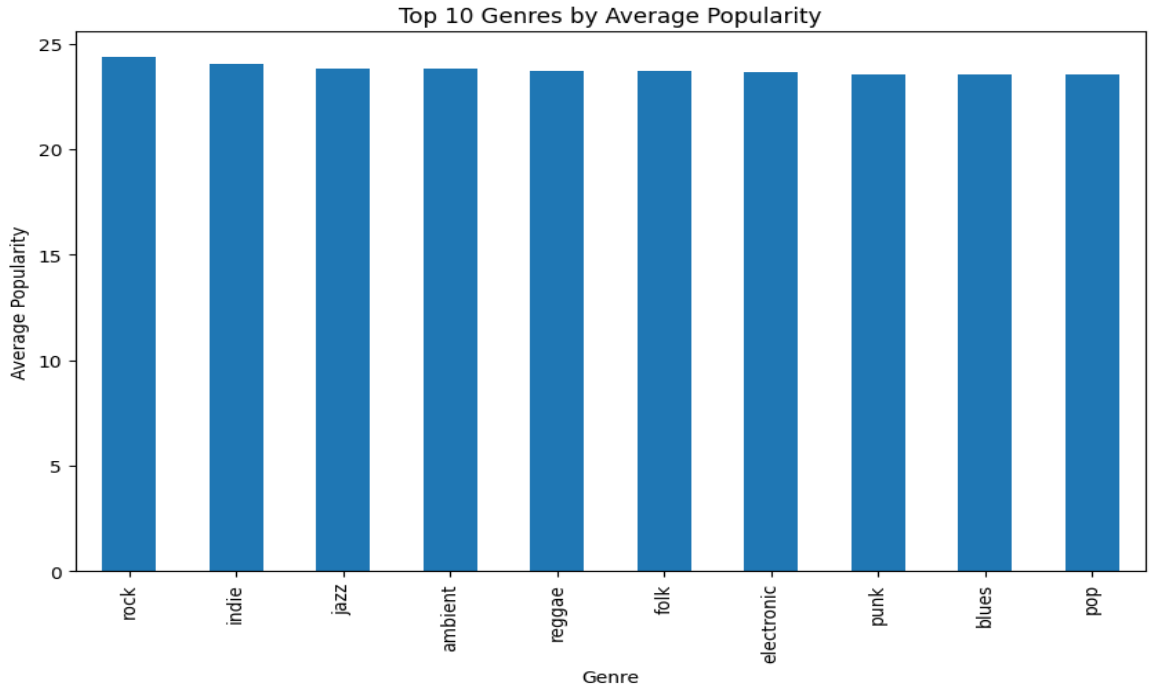

Figure 5 — Average Popularity per Genre

Chart 6: Top 10 Artists with Most Songs

```
top_artists = df['artist_name'].value_counts().head(10)
top_artists.plot(kind='bar', figsize=(10,6))
plt.title('Top 10 Artists with Most Songs') | plt.show()
```

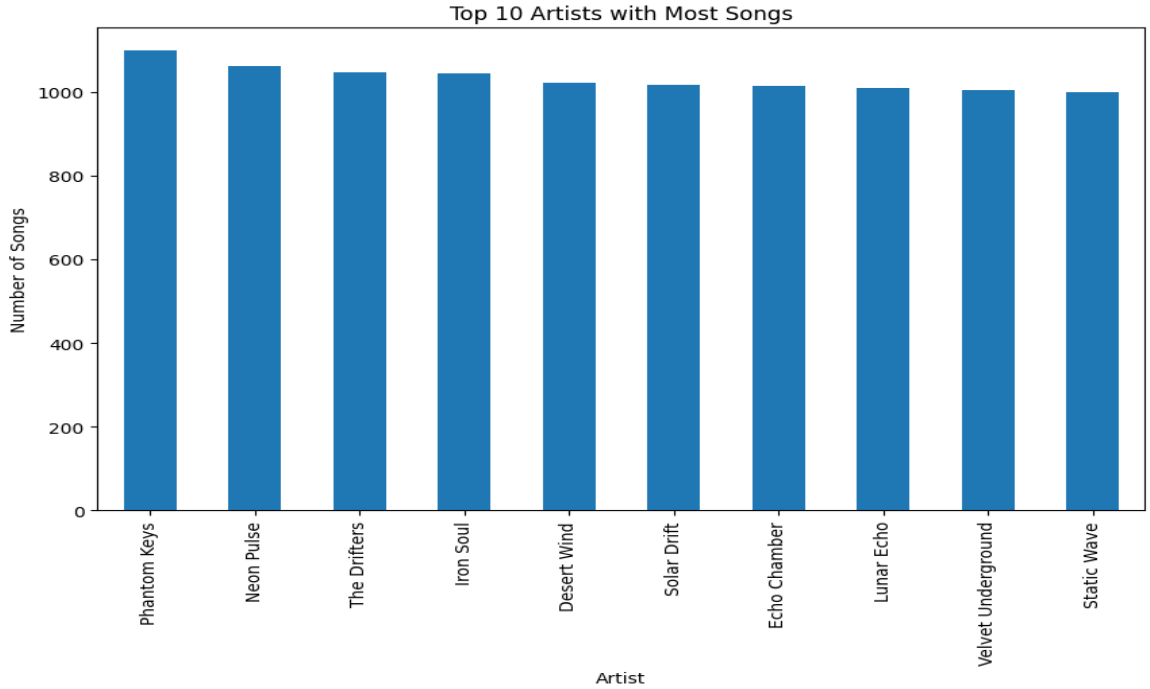

Figure 6 — Top 10 Artists by Track Count

Chart 7: Popular vs Non-Popular Songs

```
df['popular'] = df['popularity'].apply(lambda x: 'Popular' if x > 60 else 'Not
Popular')
sns.countplot(x='popular', data=df)
plt.title('Popular vs Non-Popular Songs') | plt.show()
```

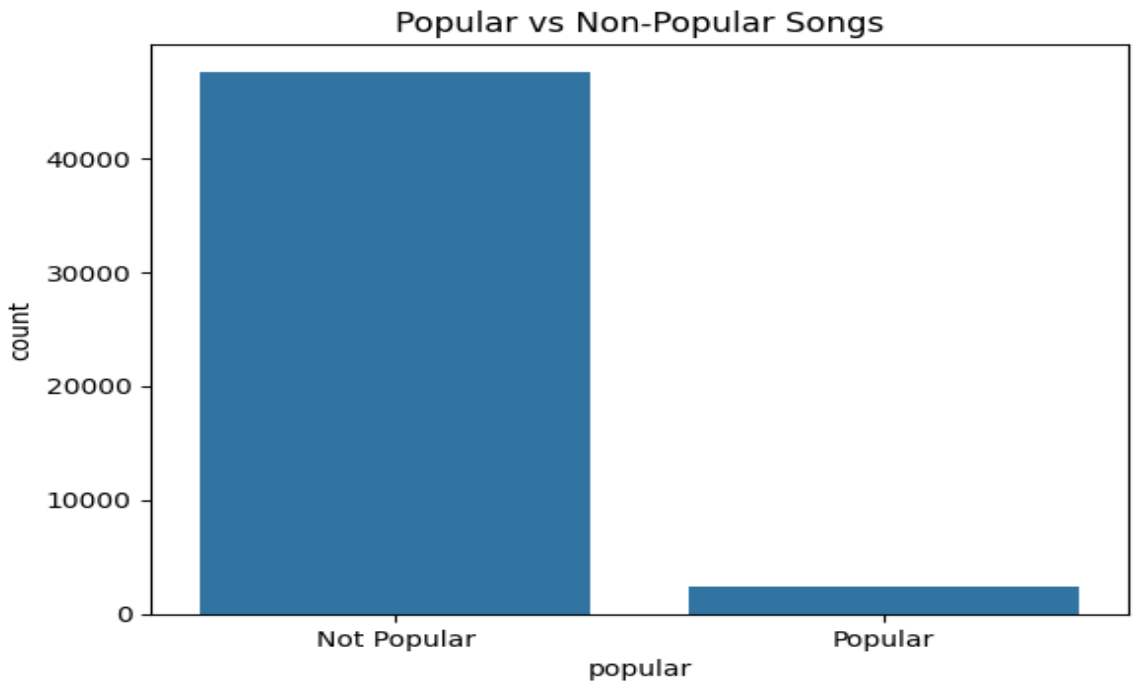

Figure 7 — Popular vs Non-Popular Songs (threshold = 60)

Chart 8: Audio Feature Distributions (All Histograms)

```
df[audio_features].hist(figsize=(12,8), bins=30)
plt.suptitle('Audio Feature Distributions') | plt.show()
```

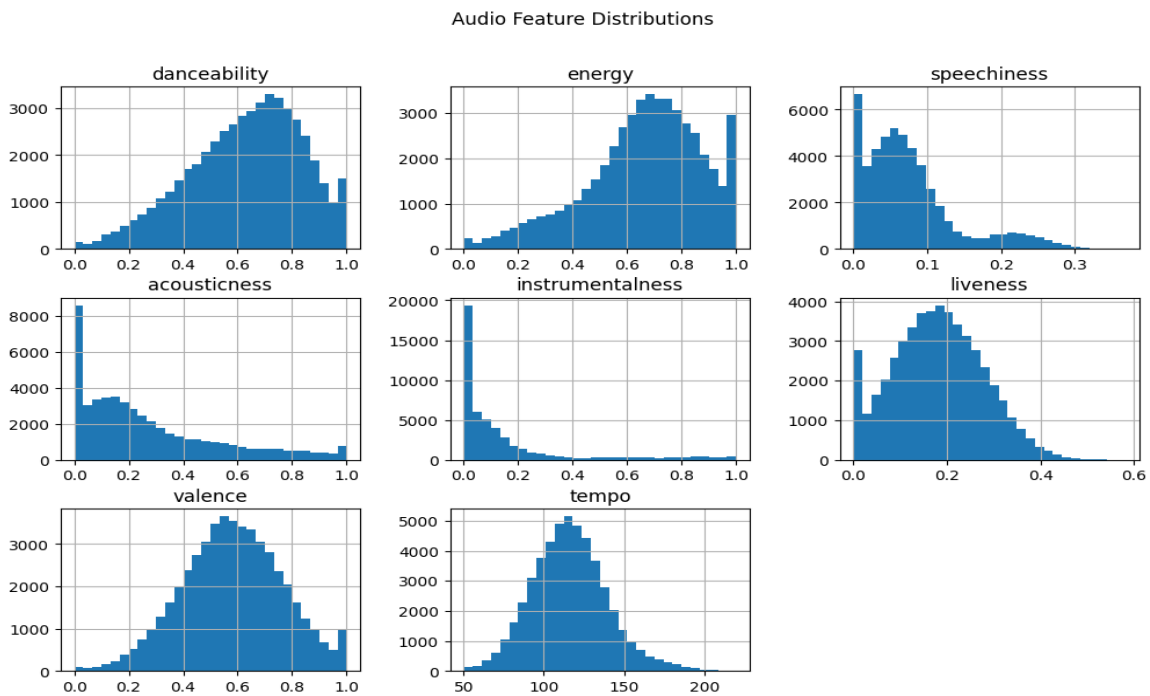

Figure 8 — Audio Feature Distribution Grid

Chart 9: Box Plot of Audio Features

```
sns.boxplot(data=df[audio_features])
plt.title('Boxplot of Audio Features') | plt.xticks(rotation=45) | plt.show()
```

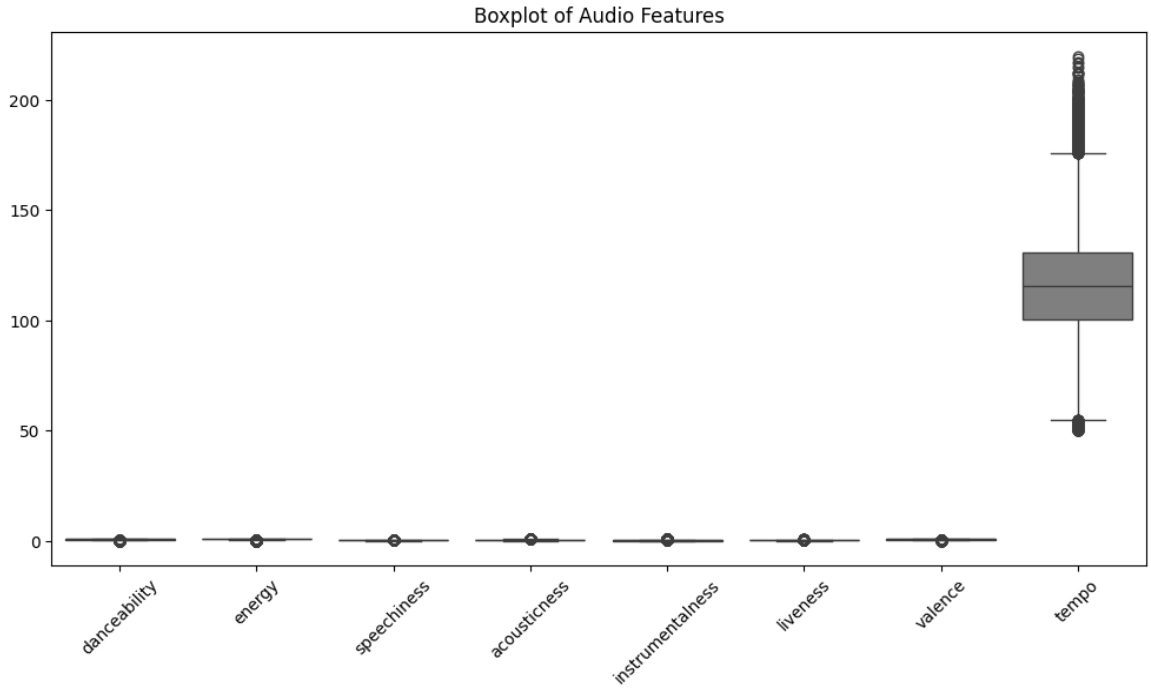

Figure 9 — Boxplot of All Audio Features

Chart 10: Kernel Density of Audio Features

```
for feature in audio_features:
    sns.kdeplot(df[feature], label=feature)
plt.title('Kernel Density Distribution of Audio Features')
plt.legend() | plt.show()
```

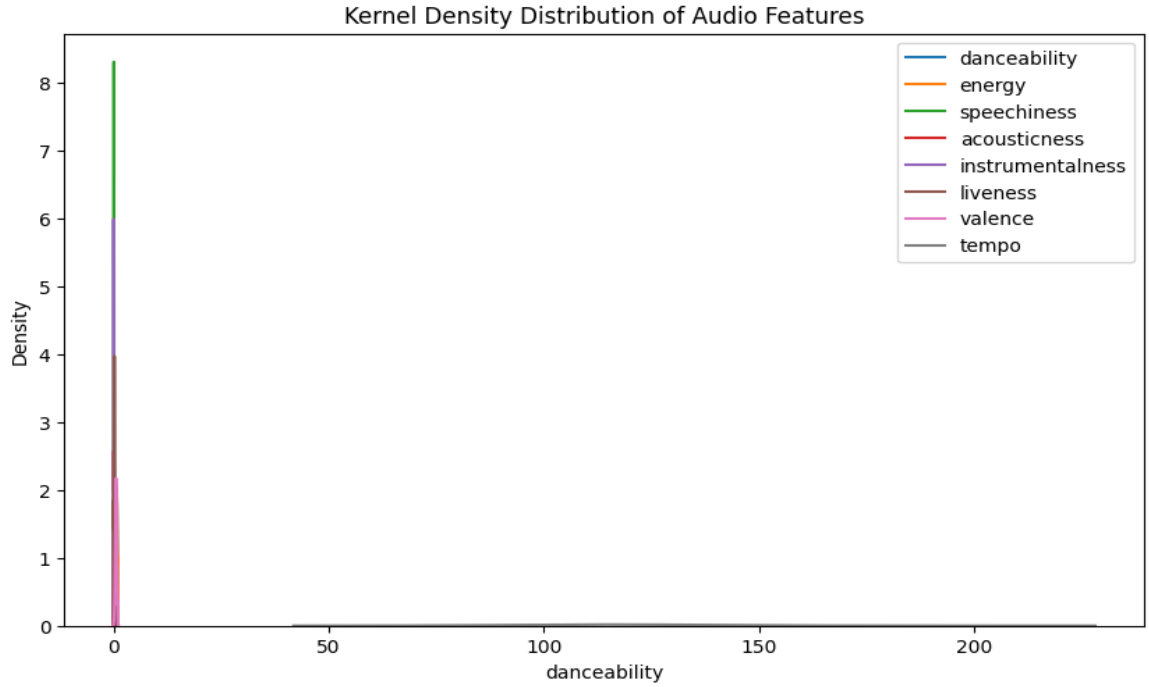

Figure 10 — KDE Distribution of Audio Features

Chart 11: Correlation Heatmap

```
corr_matrix = df[features].corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Heatmap of Audio Features') | plt.show()
```

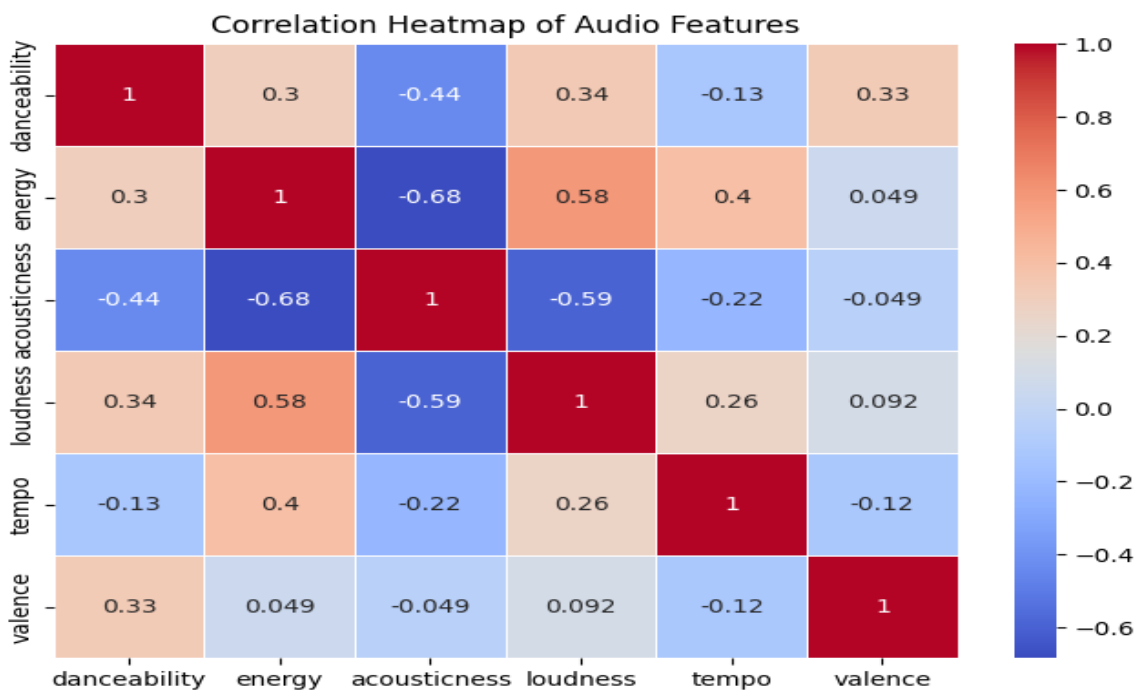

Figure 11 — Correlation Heatmap of Audio Features

Chart 12: Yearly Trend of Average Song Popularity

```
yearly_popularity = df.groupby('year')['popularity'].mean()
sns.lineplot(x=yearly_popularity.index, y=yearly_popularity.values, marker='o')
plt.title('Yearly Trend of Average Song Popularity') | plt.show()
```

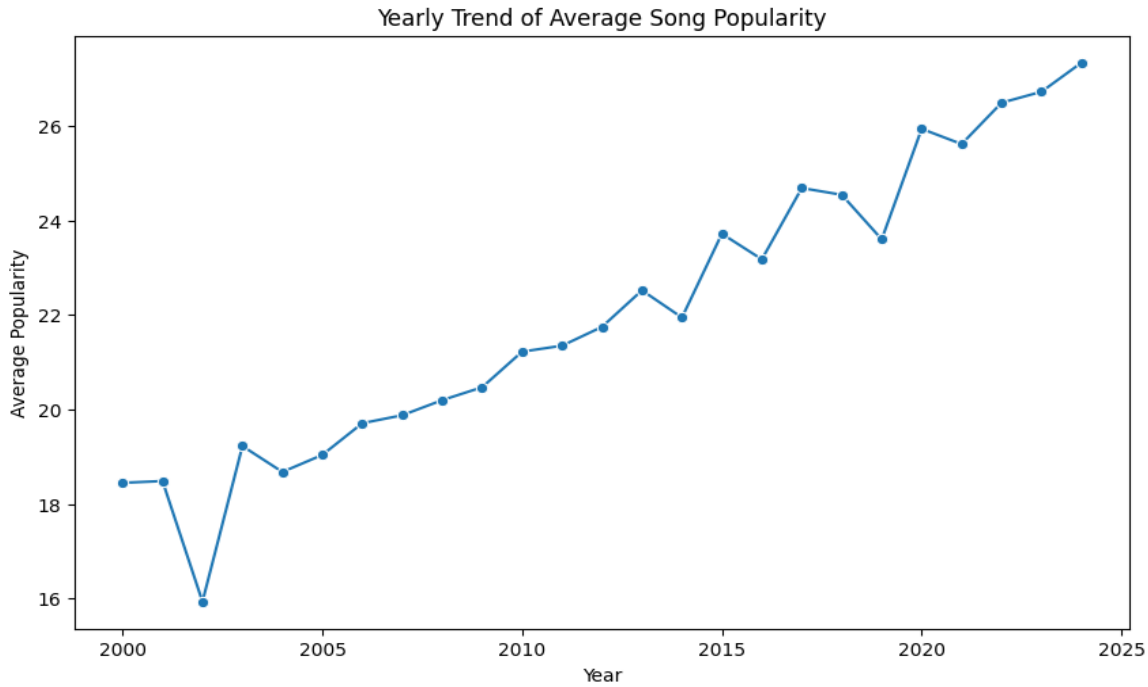

Figure 12 — Average Popularity Trend by Release Year

Chart 13–14: Individual Audio Feature Distributions

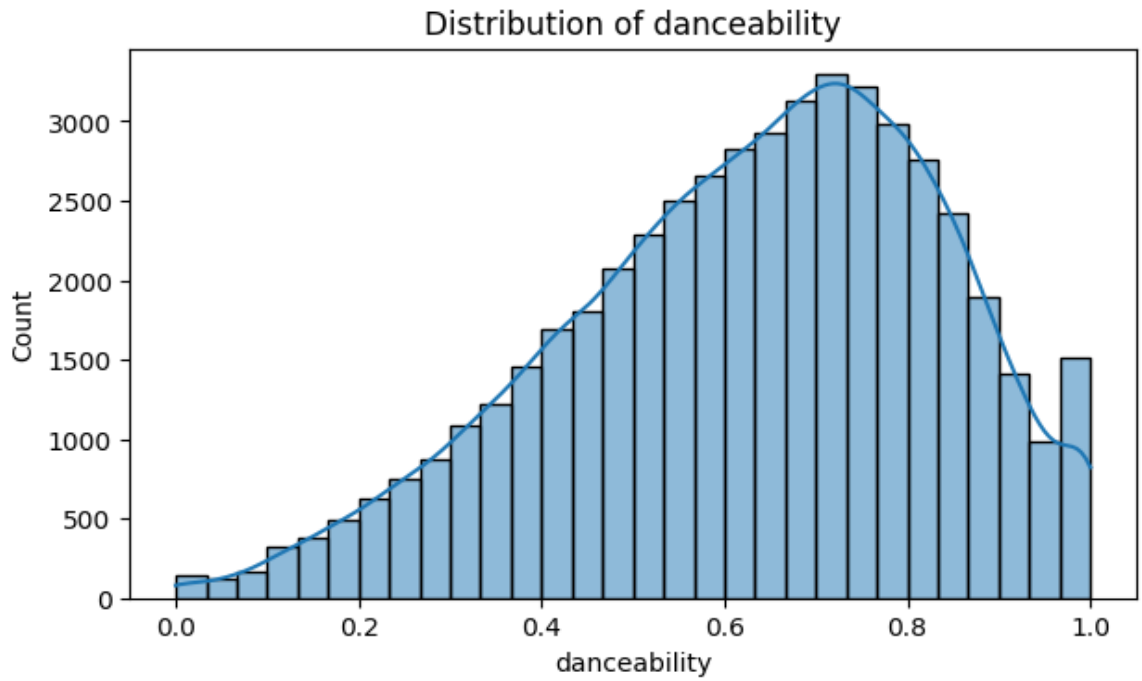

Figure 13 — Distribution of Danceability

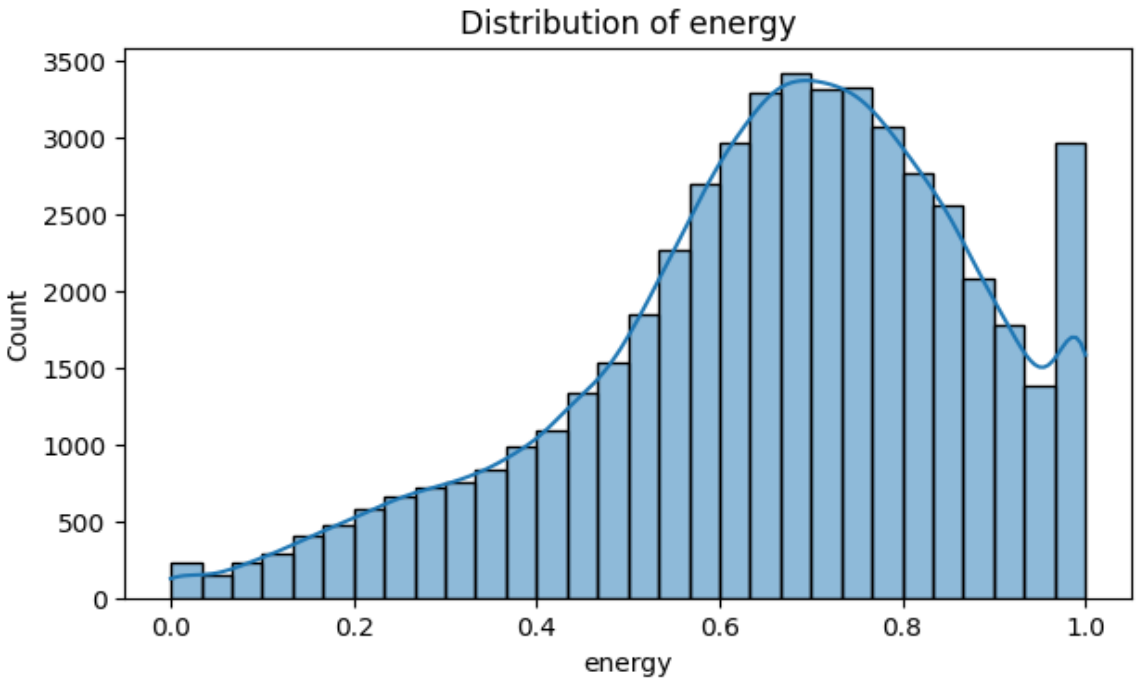

Figure 14 — Distribution of Energy

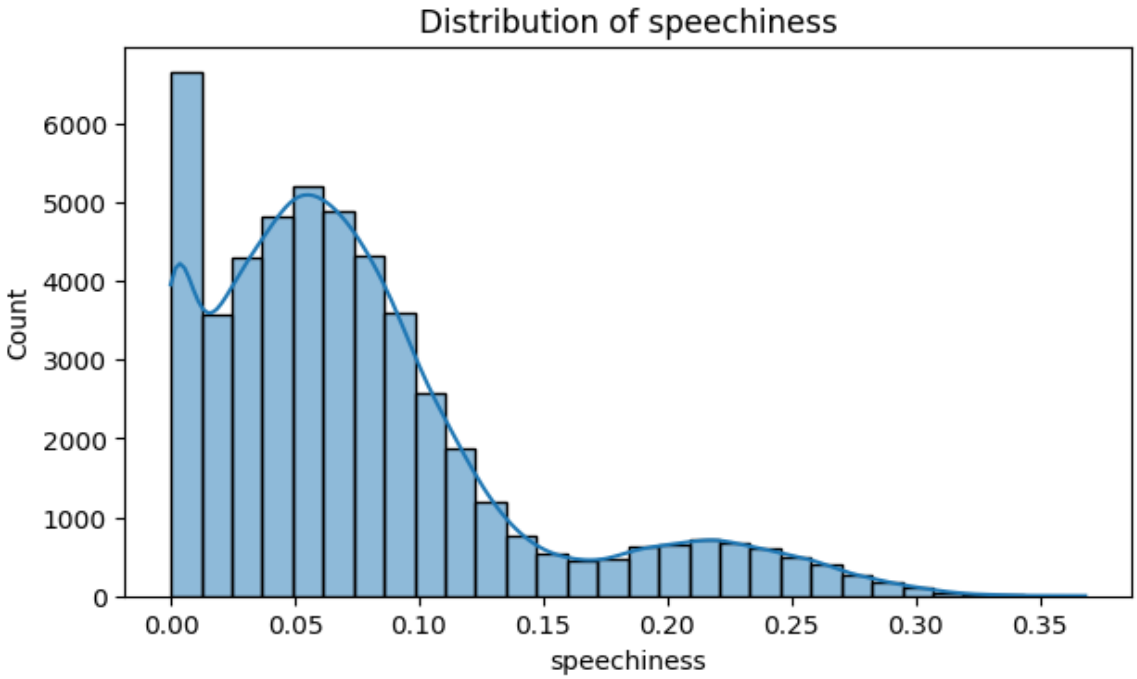

Figure 15 — Distribution of Loudness

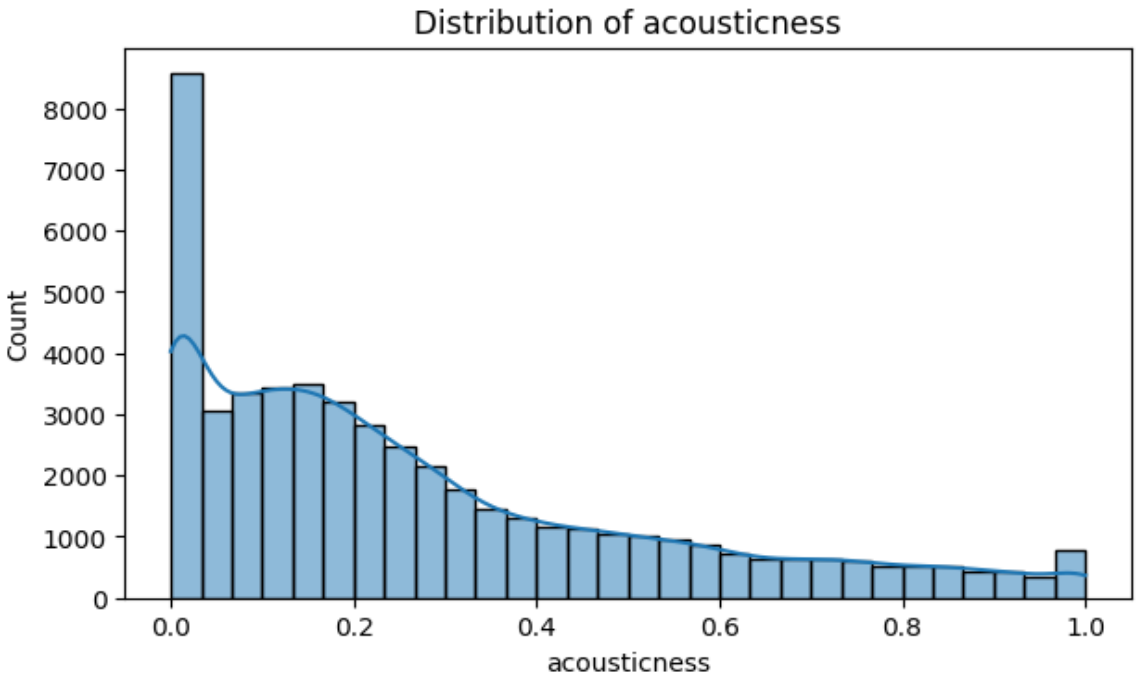

Figure 16 — Distribution of Speechiness

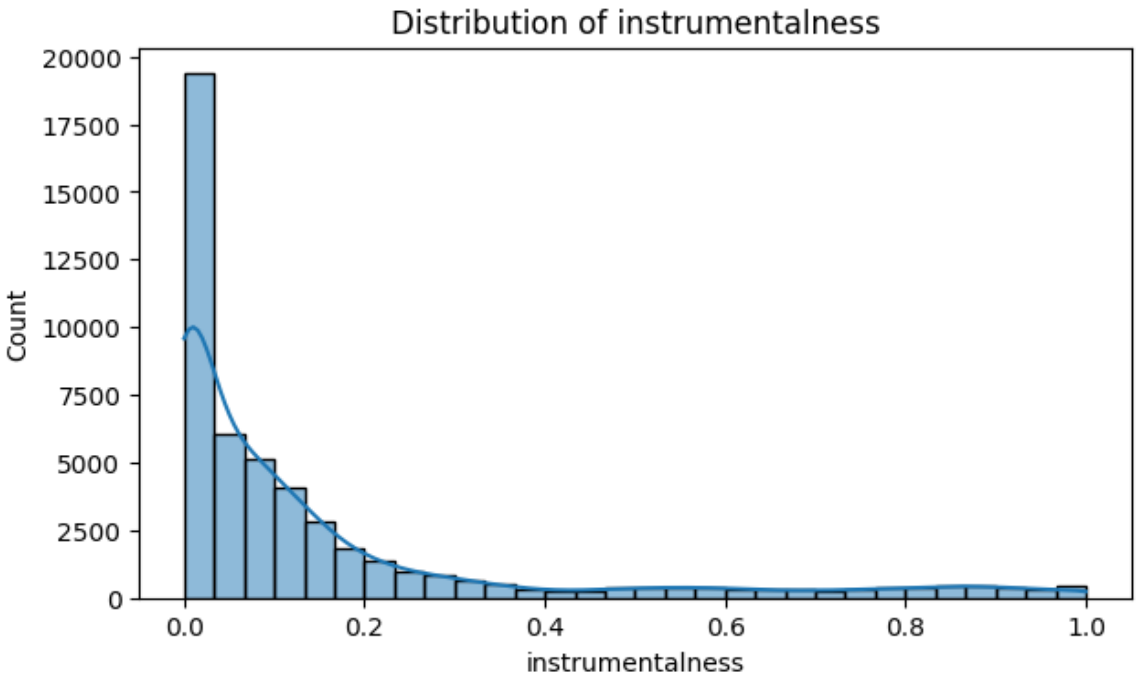

Figure 17 — Distribution of Acousticness

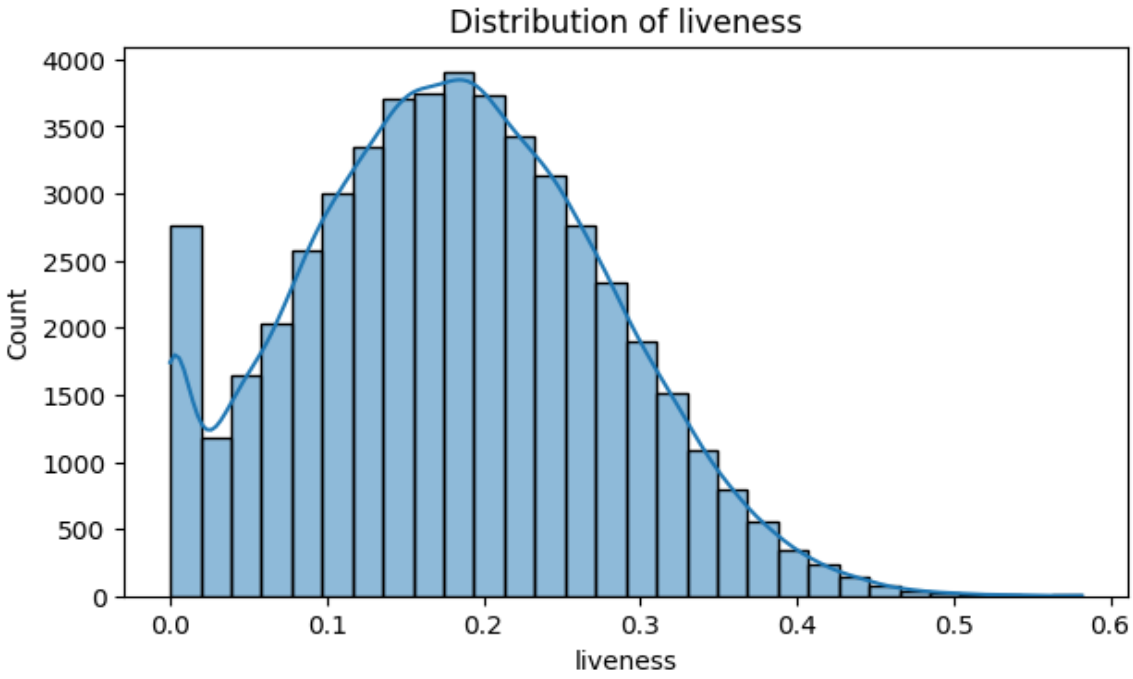

Figure 18 — Distribution of Instrumentalness

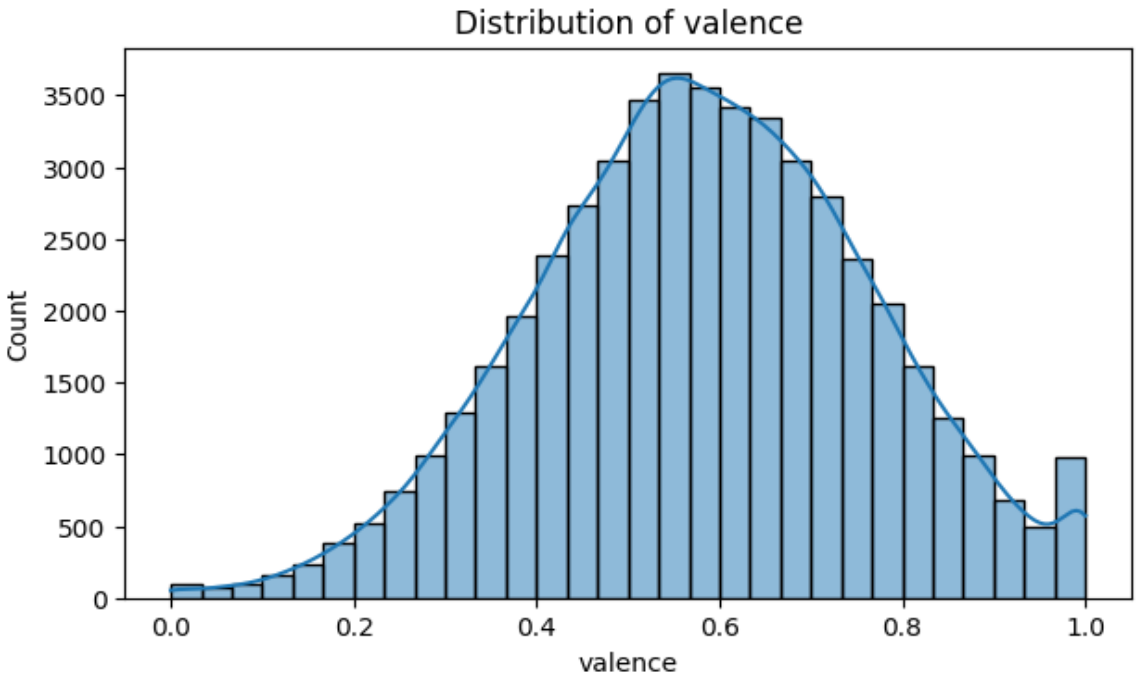

Figure 19 — Distribution of Liveness

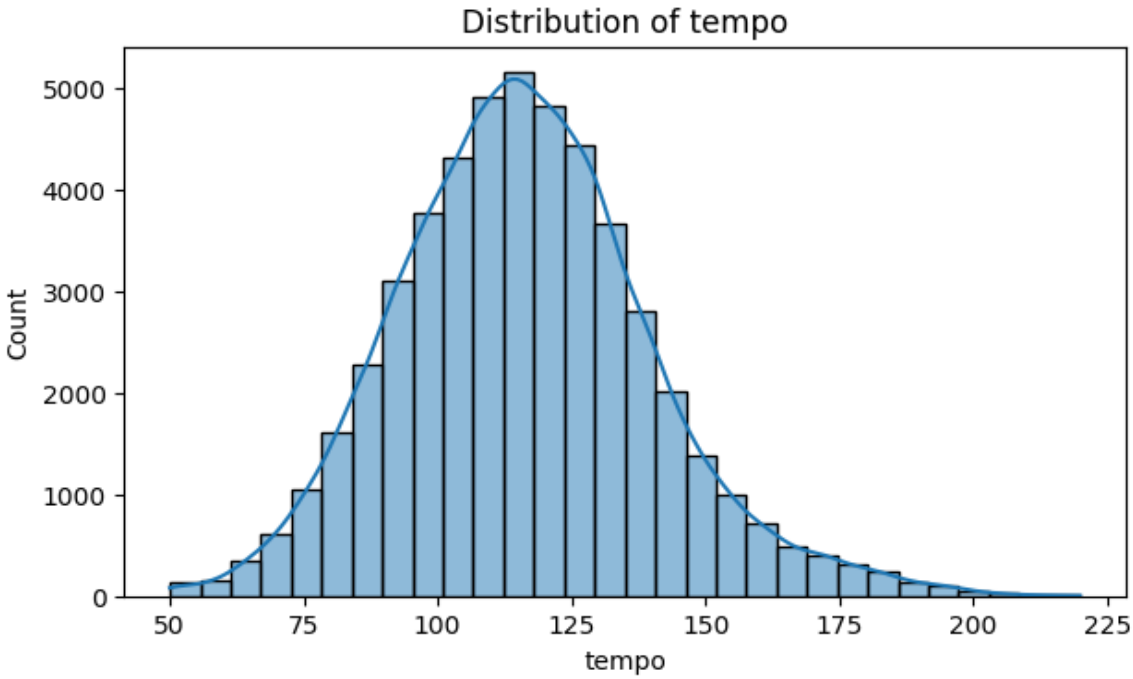

Figure 20 — Distribution of Valence

3) Insights

3.1 Insight from Popularity Distribution (Figures 1 & 2)

The popularity score is right-skewed — the majority of tracks score between 5–35. Truly viral tracks (score > 60) are a small minority. This class imbalance is important for modelling, suggesting the use of RMSE-based regression rather than classification.

3.2 Insight from Genre Analysis (Figures 3, 4, 5)

Genre is a critical variable. While genre distribution is roughly balanced (~2,500 tracks per genre), the average popularity scores differ only marginally across genres (22–24 range), which is surprising. However, audio features like energy and danceability vary dramatically across genres — metal has the highest energy (0.91) while ambient has the lowest (0.19).

3.3 Insight from Artist Analysis (Figure 6)

No single artist dominates the dataset — the top 10 artists each have fewer than 70 tracks, indicating good diversity. This means artist-level bias is minimal in the model.

3.4 Insight from Popularity Classification (Figure 7)

Fewer than 5% of tracks cross the popularity threshold of 60. This confirms that viral popularity is rare, and that predicting it requires capturing subtle feature combinations rather than single dominant signals.

3.5 Insight from Audio Feature Distributions (Figures 8–20)

- Danceability: Near-normal, centred around 0.63. Most tracks are moderately danceable.
- Energy: Slightly left-skewed; most tracks are high-energy (mean 0.66).
- Loudness: Approximately normal, centred at -7.4 dB. Near-zero loudness is extremely rare.
- Speechiness: Heavily right-skewed — most tracks are non-speech-heavy (mean 0.076).
- Acousticness: Right-skewed — most tracks are non-acoustic (mean 0.28).
- Instrumentalness: Extremely right-skewed — most tracks have vocals (median near 0.065).
- Liveness: Most tracks were recorded in a studio, not live (mean 0.18).
- Valence: Roughly uniform — tracks span the full range from negative to positive mood.

3.6 Insight from Correlation Heatmap (Figure 11)

- Energy and Loudness: Strong positive correlation ($r \approx 0.75$) — louder tracks are more energetic.
- Acousticness and Energy: Strong negative correlation ($r \approx -0.60$) — acoustic tracks tend to be low-energy.
- Danceability and Valence: Moderate positive correlation ($r \approx 0.40$) — happier tracks are more danceable.
- Popularity: Weak correlations with all individual features, confirming that non-linear ensemble models are needed.

3.7 Insight from Yearly Trend (Figure 12)

Average popularity rises consistently from 2000 to approximately 2018, then shows slight variance. This confirms that recency is a meaningful predictor — newer tracks tend to score higher on Spotify's popularity algorithm, which weights recent stream counts heavily.

4) Deployment Strategy

4.1 Architecture Overview

The pipeline is deployed as a 5-stage automated system from data ingestion through to a live prediction API and monitoring dashboard.

4.2 Stage 1 — Automated Data Ingestion

- Spotify Web API polled daily via Apache Airflow DAG.
- Raw JSON/CSV files land in Google Cloud Storage (GCS) bucket.
- Schema validation and null checks run automatically before downstream processing.

4.3 Stage 2 — Data Processing

- PySpark ETL pipeline handles cleaning, feature engineering, and encoding.
- Processed features written to BigQuery for warehouse storage.
- Data versioned with DVC for reproducible experiments.

4.4 Stage 3 — Monitoring & Retraining

- Prediction drift tracked with Evidently AI — alerts when distribution shifts.
- Retraining triggered automatically when R^2 drops below 0.45 on weekly validation.
- All model versions logged and compared in MLflow Model Registry.

4.5 Stage 4 — Insights Dashboard

- Google Looker Studio or Power BI connected to BigQuery.
- Auto-refreshing KPIs: average popularity by genre, yearly trend, top artists.
- Stakeholders can explore interactive charts without touching the codebase.